

Fiche de Révision : Classification et Arbres de Décision avec RapidMiner

1.0 Les Fondamentaux de la Classification

1.1 Introduction aux Concepts Clés

La classification est une technique fondamentale de l'apprentissage automatique qui joue un rôle central dans la prise de décision en entreprise. Son objectif principal est de transformer des données brutes en catégories claires et exploitables. En attribuant chaque donnée à une classe prédéfinie (par exemple, "spam" ou "non spam", "client potentiel" ou "non"), la classification permet d'automatiser et d'optimiser des processus, allant de la détection de fraude à la segmentation marketing.

1.2 Distinction entre Classification et Prédiction

Pour bien comprendre la classification, il est essentiel de la distinguer de la prédiction, un autre concept clé de l'analyse de données. Le tableau ci-dessous résume leurs différences fondamentales.

Critère	Classification	Prédiction
Définition	Attribuer une donnée à une classe prédéfinie.	Estimer une valeur numérique continue.
Sortie Attendue	Une étiquette (catégorie qualitative).	Une valeur numérique (quantitative).
Exemple	Prédire si un email est un spam ou non.	Prédire le chiffre d'affaires du mois prochain.

1.3 Le Processus de Construction d'un Modèle en Deux Étapes

La création d'un modèle de classification fiable suit un processus structuré en deux phases distinctes : la construction du modèle et l'évaluation de sa pertinence.

- Étape 1 : Construction du Modèle** Cette première étape utilise un sous-ensemble de données, appelé **jeu d'apprentissage (training set)**, où chaque exemple est déjà associé à une classe connue (le *label*). L'algorithme analyse ces données pour identifier des schémas et des relations. Le résultat de cette phase est un modèle, qui peut être représenté sous diverses formes, comme un ensemble de règles de classification (par exemple, une règle déduite pourrait être : SI Grade = 'Professeur' OU Années > 6 ALORS Titulaire = 'oui') ou, plus visuellement, un arbre de décision.
- Étape 2 : Estimation de la Pertinence** Une fois le modèle construit, sa performance doit être évaluée. Pour ce faire, on utilise un second sous-ensemble de données, le **jeu de test**, que le modèle n'a jamais vu auparavant. Le modèle est appliqué à ce jeu de test pour prédire les classes. La **pertinence (accuracy)** du modèle est alors calculée en comparant ses prédictions aux labels réels. Cette mesure correspond au pourcentage de tuples correctement classifiés et indique la capacité du modèle à généraliser ses apprentissages à de nouvelles données.

1.4 Apprentissage Supervisé vs. Non Supervisé

La classification s'inscrit dans un cadre plus large de l'apprentissage automatique. Il est important de la situer par rapport à d'autres approches.

- Apprentissage Supervisé (Classification)** : Dans ce type d'apprentissage, les données d'entraînement sont accompagnées de labels connus. L'objectif est d'entraîner un modèle capable de prédire correctement le label pour de nouvelles données, en se basant sur les exemples vus dans le training set. La classification est l'exemple par excellence de l'apprentissage supervisé.
- Apprentissage Non Supervisé (Regroupement)** : Ici, les labels des données ne sont pas connus à l'avance. L'objectif n'est pas de prédire une classe prédéfinie, mais de découvrir des structures ou des groupes inhérents aux données elles-mêmes. Cette technique est souvent utilisée pour la segmentation de clientèle ou la détection d'anomalies. Parmi les techniques d'apprentissage supervisé, l'algorithme des arbres de décision est l'un des plus populaires et des plus intuitifs, ce que nous allons explorer en détail.

2.0 Analyse Approfondie des Arbres de Décision

2.1 Introduction à la Structure et à l'Utilité des Arbres de Décision

Un arbre de décision est un outil de classification puissant qui modélise une série de décisions sous la forme d'une structure arborescente. Sa grande force réside dans son **interprétabilité** : il transforme des règles de décision potentiellement complexes en un diagramme visuel et facile à comprendre, imitant le raisonnement humain. Chaque branche de l'arbre représente une question sur un attribut, menant finalement à une conclusion sur la classe à attribuer.

2.2 Anatomie d'un Arbre de Décision

Un arbre de décision est composé de trois éléments fondamentaux :

- **Nœuds internes** : Représentent les tests effectués sur les attributs des données (par exemple, "L'âge est-il inférieur à 30 ?").
- **Arcs** : Correspondent aux résultats possibles des tests (par exemple, "Oui" ou "Non"). Chaque arc mène à un nœud suivant.
- **Nœuds feuilles** : Représentent les classes finales prédites par le modèle (par exemple, "Achète l'ordinateur").

2.3 Le Processus de Construction de l'Arbre

La construction d'un arbre de décision suit une approche "top-down récursive", ce qui signifie que l'arbre est construit du sommet (la racine) vers les feuilles de manière itérative.

1. **Initialisation** : Au départ, toutes les données (tuples) du jeu d'apprentissage sont regroupées à la racine de l'arbre.
2. **Pré-requis des Attributs** : L'algorithme fonctionne de manière native avec des attributs qualitatifs. Si des attributs numériques sont utilisés, une étape de discrétisation (transformation en catégories, ex: âge < 30) est souvent appliquée en amont.
3. **Partitionnement Récursif** : À chaque étape, les tuples d'un nœud sont séparés en sous-ensembles plus petits en fonction de la valeur d'un attribut de test.
4. **Choix de l'Attribut** : Le choix de l'attribut de test à chaque nœud est crucial. Il est guidé par des heuristiques, comme le **gain informationnel**, qui visent à sélectionner l'attribut qui sépare le mieux les classes et maximise la "pureté" des sous-ensembles résultants.
5. **Conditions d'Arrêt** : Le processus de partitionnement s'arrête pour une branche lorsque tous les tuples d'un nœud appartiennent à la même classe. Ce nœud devient alors une feuille.

2.4 L'Importance du Choix de l'Attribut de Partitionnement

Le choix du premier attribut à la racine de l'arbre a un impact direct sur sa simplicité et son efficacité. Un bon choix mène à un arbre compact, tandis qu'un mauvais choix le rend inutilement complexe. Illustrons ce point avec le jeu de données suivant :

A	B	Classe
0	1	C1
0	0	C1
1	1	C2
1	0	C2

1. **Cas 1 : Partitionnement avec l'attribut A** L'arbre est construit en une seule étape. Le nœud racine teste A. Si $A=0$, les deux exemples restants sont de classe C1. Si $A=1$, les deux exemples restants sont de classe C2. L'arbre est simple et efficace, car chaque branche mène directement à une feuille pure.
2. **Cas 2 : Partitionnement avec l'attribut B** L'arbre est plus complexe. Le nœud racine teste B. La branche $B=0$ contient un mélange des classes C1 et C2. De même, la branche $B=1$ contient un mélange de C1 et C2. Il est donc nécessaire d'ajouter un second niveau de partitionnement sur l'attribut A pour chaque branche afin de séparer les classes. L'arbre final est plus profond et moins efficace.

Cet exemple illustre la nécessité d'une méthode quantitative pour choisir le meilleur attribut. C'est ici qu'intervient la théorie de l'information.

2.5 Notion de Théorie de l'Information et Entropie

Pour résoudre le problème du choix de l'attribut optimal, comme vu avec l'exemple des attributs A et B, les algorithmes s'appuient sur le concept d'**entropie**, issu de la théorie de l'information. L'entropie est une mesure de l'incertitude ou de l'impureté d'un ensemble de données.

- Un ensemble **pur** (où tous les éléments appartiennent à la même classe) a une entropie de 0 (aucune incertitude).
- Un ensemble **impur** (avec un mélange équilibré de plusieurs classes) a une entropie maximale.

L'objectif est de choisir l'attribut qui **réduit le plus l'entropie**, c'est-à-dire celui qui apporte le plus d'information. Mathématiquement, pour un ensemble S contenant p exemples positifs et n exemples négatifs, l'entropie se calcule ainsi : $H(p, n) = -p/(p+n) \log_2(p/(p+n)) - n/(p+n) \log_2(n/(p+n))$

Comme l'illustre l'analogie du temps dans le désert, apprendre qu'il fera beau demain est une information de faible valeur car c'est très probable. En revanche, apprendre qu'il va pleuvoir est une information de grande valeur. De même, un attribut qui sépare nettement les classes apporte beaucoup d'information et est donc un bon candidat pour le partitionnement.

2.6 Extraction des Règles de Classification

L'un des principaux avantages des arbres de décision est la facilité avec laquelle on peut en extraire des règles de classification lisibles sous la forme **SI-ALORS** (IF-THEN). Chaque chemin unique de la racine de l'arbre à un nœud feuille constitue une règle. Les conditions testées sur les nœuds internes le long de ce chemin sont combinées avec des opérateurs ET pour former la partie SI de la règle, tandis que la classe du nœud feuille constitue la partie ALORS.

Bien que puissants, les arbres de décision peuvent être sujets à un problème courant appelé le surapprentissage.

3.0 Gérer le Surapprentissage (Overfitting)

3.1 Définition et Conséquences du Surapprentissage

Le surapprentissage (overfitting) se produit lorsqu'un modèle de classification apprend "trop bien" les données d'apprentissage. Au lieu de capturer les tendances générales, il mémorise les détails, le bruit et les anomalies spécifiques au jeu d'entraînement. En conséquence, bien qu'il puisse atteindre une précision parfaite sur ces données, ses performances s'effondrent lorsqu'il est confronté à de nouvelles données, car il a perdu sa capacité à généraliser.

3.2 Évaluation des Risques

Un modèle en surapprentissage présente plusieurs risques majeurs :

- **Apprentissage excessif du bruit** : Le modèle intègre des exceptions et des anomalies comme s'il s'agissait de règles générales.
- **Complexité trop élevée** : L'arbre devient excessivement grand et détaillé, avec des branches qui ne s'appliquent qu'à quelques cas très spécifiques.

- **Perte de la capacité de généralisation** : La performance du modèle sur un jeu de test est médiocre, le rendant inutilisable en pratique.

Face à ces risques, il est impératif d'employer des stratégies pour simplifier le modèle. La principale technique pour y parvenir est l'élagage, ou *pruning*.

3.3 Stratégies de Généralisation : Le Pruning (Élagage)

Pour éviter le surapprentissage, on cherche à simplifier l'arbre pour qu'il généralise mieux. Cette simplification est appelée **élagage (pruning)** et peut être réalisée via deux approches principales.

Approche	Méthode	Avantage	Inconvénient
Prepruning	Arrêter la construction de l'arbre prématurément si le gain informationnel d'un nouveau découpage est inférieur à un seuil prédéfini.	Rapide à construire, l'arbre est directement plus simple.	Le choix du seuil optimal est difficile et peut stopper la construction trop tôt, sacrifiant de la précision.
Postpruning	Construire l'arbre complet, puis supprimer les branches qui n'améliorent pas significativement la performance sur un jeu de validation.	L'arbre final est souvent plus robuste et généralise mieux.	Le processus est plus long car il nécessite la construction complète de l'arbre et des tests supplémentaires.

Exemple concret de Postpruning : Un arbre complet pourrait contenir une règle très spécifique comme SI âge ≤ 30 ET étudiant = oui ET revenu = moyen ALORS achète_ordinateur = oui. Si la condition revenu = moyen n'améliore pas significativement la performance sur le jeu de validation, elle est supprimée, résultant en une règle plus générale et robuste : SI âge ≤ 30 ET étudiant = oui ALORS achète_ordinateur = oui.

Au-delà de l'optimisation des arbres de décision, il existe d'autres familles d'algorithmes de classification qui offrent des approches alternatives pour résoudre ces problèmes.

4.0 Au-delà des Arbres de Décision : Autres Classifieurs

4.1 Introduction aux Méthodes Alternatives

Bien que les arbres de décision soient un excellent point de départ, de nombreuses autres familles d'algorithmes de classification existent. Des méthodes comme la classification bayésienne, qui s'appuie sur les probabilités, ou les méthodes d'ensemble, qui combinent plusieurs modèles, offrent des perspectives différentes et souvent des performances supérieures pour résoudre les problèmes de classification.

4.2 La Classification Bayésienne

La classification bayésienne est une approche probabiliste pour la classification.

- **Principe** : Elle est fondée sur la **théorie des probabilités et le théorème de Bayes**, qui permet de calculer la probabilité d'une hypothèse en fonction de nouvelles données. Cette approche repose sur le théorème de Bayes : $P(\text{Classe} \mid \text{Données}) = \frac{P(\text{Données} \mid \text{Classe}) \times P(\text{Classe})}{P(\text{Données})}$.
- **Fonctionnement** : Pour une nouvelle donnée, le modèle calcule la probabilité de son appartenance à chaque classe possible. Il attribue ensuite la classe qui obtient la probabilité la plus élevée.
- **Importance** : Les classifieurs bayésiens servent souvent de **référence** pour évaluer la performance d'autres méthodes. Une variante populaire et efficace est le **classifieur naïf bayésien**, qui simplifie les calculs en supposant que les attributs sont indépendants les uns des autres.

4.3 Les Forêts Aléatoires (Random Forest)

Une forêt aléatoire est un exemple de méthode d'ensemble qui s'appuie sur la puissance des arbres de décision. Il s'agit d'un **ensemble de plusieurs arbres de décision** où chacun est entraîné sur un sous-ensemble légèrement différent des données. Pour classer une nouvelle donnée, chaque arbre de la forêt "vote" pour une classe, et la classe qui reçoit la majorité des votes est retenue comme prédiction finale. Cette approche collaborative permet d'obtenir une **précision augmentée** et de réduire le risque de surapprentissage par rapport à un arbre de décision unique.

La section suivante illustre concrètement comment implémenter et comparer un arbre de décision et une forêt aléatoire dans l'outil RapidMiner.

5.0 Mise en Pratique : Classification avec RapidMiner sur le Jeu de Données Titanic

5.1 Contexte du Tutoriel

Cette section vise à appliquer les concepts théoriques vus précédemment à l'aide de l'outil RapidMiner. L'objectif est de construire un modèle capable de prédire la survie des passagers du Titanic. Pour cela, nous utiliserons l'attribut "Survived" comme variable cible, en lui assignant le rôle spécial de **Label**.

5.2 Étape 1 : Construire un Premier Arbre de Décision

La première étape consiste à créer un modèle de base pour visualiser les facteurs de survie.

1. **Définir la Cible** : On utilise l'opérateur Set Role pour désigner l'attribut « Survived » comme le Label que le modèle doit apprendre à prédire.
2. **Construire l'Arbre** : On applique l'opérateur Decision Tree avec ses paramètres par défaut, notamment une profondeur maximale de 10.
3. **Première Observation** : L'arbre généré identifie immédiatement l'attribut « Sex » (le sexe du passager) comme le facteur le plus discriminant au premier niveau, c'est-à-dire celui qui sépare le mieux les survivants des non-survivants.

5.3 Étape 2 : Simplifier et Interpréter l'Arbre

L'arbre initial, avec une profondeur de 10, est souvent trop complexe pour être facilement interprétable. Pour le rendre plus lisible, on peut ajuster ses paramètres.

- **Action** : Réduire la maximal depth (profondeur maximale) de l'arbre, par exemple à 4, puis à 2. Cette action simplifie radicalement la structure de l'arbre, ne conservant que les règles de décision les plus importantes et le rendant plus facile à analyser.

5.4 Étape 3 : Évaluer la Performance du Modèle

Pour évaluer objectivement la fiabilité de l'arbre, il est nécessaire de le tester sur des données qu'il n'a pas utilisées pour son apprentissage.

1. **Partitionner les Données** : L'opérateur Split est utilisé pour diviser le jeu de données Titanic en deux parties : 80% pour l'apprentissage du modèle et 20% pour le test.
2. **Appliquer le Modèle** : L'opérateur Apply Model prend l'arbre appris sur les 80% de données et l'applique aux 20% de données de test pour générer des prédictions.
3. **Mesurer la Performance** : L'opérateur Performance (Classification) est ensuite utilisé pour comparer les prédictions aux véritables labels du jeu de test. Il génère une matrice de confusion (tableau de vérité) et des métriques clés comme :
 - **Class Recall** : Parmi les vrais positifs (ex: les vrais survivants), quel pourcentage le modèle a-t-il correctement identifié ?
 - **Class Precision** : Parmi toutes les prédictions positives du modèle (ex: tous ceux prédits comme survivants), quel pourcentage était correct ?

5.5 Étape 4 : Analyser le Compromis Complexité vs. Précision

L'analyse de la performance révèle une relation directe entre la complexité de l'arbre et sa précision.

- **Constat** : En augmentant la maximal depth de l'arbre, les prédictions deviennent généralement plus précises sur le jeu de test.
- **Conclusion** : La conclusion est donc qu'il est souvent plus opportun d'opter pour un arbre plus complexe et difficile à lire mais plus efficace, plutôt qu'un arbre simple à déchiffrer mais qui est source d'erreurs.

5.6 Étape 5 : Comparer avec un Modèle de Forêt Aléatoire

Pour tenter d'améliorer encore la précision, on peut comparer l'arbre de décision à un modèle plus avancé comme la forêt aléatoire.

1. **Créer le Modèle Alternatif** : Le processus est simplement modifié en remplaçant l'opérateur Decision Tree par l'opérateur Random Forest.
2. **Comparer les Performances** : On répète le processus d'évaluation et on compare les métriques de performance de la forêt aléatoire à celles de l'arbre de décision. En théorie, la forêt aléatoire devrait offrir une meilleure précision globale.
3. **Analyse Fine** : Une analyse plus poussée pourrait consister à compter les cas où les deux modèles sont d'accord ou en désaccord dans leurs prédictions. Cette étape, mentionnée dans le tutoriel comme "à compléter", permet de comprendre en détail où chaque modèle excelle ou échoue.

6.0 Synthèse des Notions Clés

Cette fiche de révision a couvert les concepts essentiels de la classification, depuis les fondements théoriques jusqu'à la mise en œuvre pratique. Nous avons exploré la distinction entre classification et prédiction, le fonctionnement des arbres de décision, les défis comme le surapprentissage et les stratégies pour y remédier. L'application pratique sur RapidMiner a démontré comment construire, évaluer et comparer des modèles comme l'arbre de décision et la forêt aléatoire. La maîtrise de ces deux facettes — la théorie sous-jacente et l'application pratique via des outils dédiés — est indispensable pour mener une analyse de données efficace et produire des modèles prédictifs robustes et fiables.